



INSTITUTO POLITÉCNICO NACIONAL
ESCUELA SUPERIOR DE CÓMPUTO

UNIDAD DE APRENDIZAJE: ARQUITECTURA DE COMPUTADORAS

CONCEPTOS VERILOG

Alumno:

Areli Alejandra Guevara Badillo

Grupo 5CM2

Prof. Gelacio Castillo Cabrera

viernes, 4 de abril de 2025

INTRODUCCIÓN

Verilog es un **lenguaje de descripción de hardware (HDL)** que juega un papel esencial en el diseño de circuitos digitales y sistemas electrónicos modernos.

A diferencia de lenguajes de programación tradicionales, como C o Python, que se enfocan en instrucciones secuenciales, Verilog se centra en describir cómo se conectan y comportan los componentes electrónicos. Esto incluye desde operaciones básicas, como sumar números, hasta sistemas completos que procesan información en tiempo real. Por ejemplo, Verilog se utiliza en el diseño de procesadores, sistemas de comunicación y dispositivos IoT (Internet de las Cosas), donde la precisión y la eficiencia son críticas.

En esta tarea, se exploran dos aspectos fundamentales de Verilog:

1. **Los bloques básicos** que componen un archivo implementable, como módulos, variables y estructuras de control.
2. **Los niveles de abstracción**, que permiten adaptar el diseño a diferentes etapas del desarrollo, desde una visión general (como un algoritmo) hasta detalles técnicos (como transistores individuales).

Estos conceptos no solo ayudan a organizar el código, sino que también optimizan el proceso de simulación y verificación, asegurando que el circuito funcione correctamente antes de fabricarse físicamente. Además, Verilog admite combinar múltiples niveles de abstracción en un mismo proyecto, lo que se conoce como **Modelado de Niveles Mixtos**. Esta capacidad es especialmente útil en proyectos grandes, donde algunos componentes pueden requerir descripciones detalladas, mientras que otros se modelan de manera simplificada.

El objetivo de este reporte es proporcionar una guía clara y accesible sobre cómo estructurar diseños en Verilog y cómo elegir el nivel de abstracción adecuado según las necesidades del proyecto. Con esto, se busca sentar las bases para que futuros diseñadores puedan crear sistemas digitales eficientes y confiables.

BLOQUES BÁSICOS DE UN ARCHIVO VERILOG

Los archivos Verilog se construyen a partir de componentes esenciales que permiten describir, simular y sintetizar circuitos digitales. A continuación, se explican estos bloques de manera sencilla, con ejemplos prácticos:

1. Módulos y Puertos

Los módulos son como "cajas" que encapsulan una parte del circuito. Los módulos son las entidades principales de diseño, y los puertos son las interfaces que permiten la comunicación entre los módulos.

Cada módulo tiene **puertos** (entradas y salidas) para comunicarse con otros módulos.

Los puertos permiten especificar cómo los datos y las señales pueden entrar o salir de un módulo.

- Se definen con *module* y terminan con *endmodule*.
- Los puertos se declaran input (entrada), output (salida) o inout (bidireccional).

Ejemplo:

```
module sumador (  
    input a, b,           // Entradas  
    output suma           // Salida  
);  
    assign suma = a + b;  // Lógica del sumador  
endmodule
```

2. Declaraciones de Variables

Para declarar una variable en Verilog, se usa la sintaxis `<TIPO> [<MSB> : <LSB>] <NOMBRE>;`

Los tipos de variables en Verilog se usan para almacenar valores.

Los tipos de datos en Verilog representan elementos de almacenamiento de datos y elementos de transmisión.

Tipos principales:

- `wire`: Representa cables que conectan componentes. No almacenan valores.
- `reg`: Almacena valores temporalmente, como una memoria. Se usa en bloques *always* o *initial*.

Ejemplo:

```
int i = 0;           // variable con iniciador
wire conexion;      // cable que une dos componentes
reg contador;       // variable que guarda un valor
```

3. Flujo de Datos (Dataflow)

Es un estilo de modelado que se utiliza para describir la función lógica de circuitos combinatorios. Se basa en sentencias de asignación continua y se utiliza para modelar expresiones combinatorias.

Describe cómo se mueven los datos entre cables usando operadores lógicos (AND, OR, etc.).

- **Sintaxis clave:** `assign` para asignaciones continuas.

Ejemplo:

```
assign resultado = entrada1 & entrada2;
```

4. Instanciación de Módulos

Reutilizar módulos dentro de otros módulos, como armar un circuito con piezas preconstruidas. Esto permite crear sistemas complejos combinando módulos más simples.

Características de la instanciación de módulos

- Cada instancia es una copia independiente del módulo original.
- Cada instancia puede tener sus propias entradas, salidas, y lógica interna.
- Las instancias interactúan entre sí mediante conexiones de módulos.
- La instanciación de módulos promueve la reutilización del código.
- Permite construir sistemas complejos de forma eficiente.

Ejemplo:

```
module sistema (
    input x, y,
    output total
);
    // Instanciar el módulo sumador
    sumador mi_sumador (
        .a(x),
        .b(y),
        .suma(total)
    );
endmodule
```

5. Bloques de Comportamiento (always e initial)

El código de comportamiento se usa para describir el comportamiento de un diseño a un nivel superior, para simularlo

- **always:** Ejecuta código continuamente, como responder a cambios en una señal (ej: un reloj).
- **initial:** Ejecuta código una sola vez al inicio de la simulación (útil para pruebas).

Ejemplo:

```
always @(posedge clk) begin
    if (reset)
        contador <= 0;
    else
        contador <= contador + 1;
end
```

6. Tareas y Funciones

Son subprogramas que se usan para dividir el código en partes más pequeñas y reutilizarlo.

Tareas (task):

- Grupo de instrucciones que pueden incluir retardos.
- Son un tipo de subrutina que puede consumir tiempo.
- Se declaran y definen al igual que las funciones.
- No tienen un tipo de retorno, sino que usan un puerto con la dirección de salida en los argumentos.
- Ejecutan varias sentencias secuenciales, pero no devuelven ningún valor.
- Pueden tener un número ilimitado de salidas.

Funciones (function):

- Devuelven un valor y no usan retardos.
- Son equivalentes a la lógica combinatoria.
- Realizan un cálculo y devuelven un único valor.
- No pueden utilizarse para reemplazar código que contenga operadores de control de eventos o retardos.

Ejemplo:

```
function [7:0] multiplicar;
    input [3:0] num1, num2;
    begin
        multiplicar = num1 * num2;
    end
endfunction
```

7. Bloque de Diseño

Los bloques de diseño incluyen módulos, bloques de especificación, bloques always, bloques generate, y bloques iniciales. Estos bloques permiten definir componentes, especificar comportamientos, y automatizar tareas.

El núcleo del circuito. Define entradas, salidas y la lógica interna.

Ejemplo:

```
module semaforo (
```

```

    input  sensor,
    output verde, rojo
);
    assign verde = sensor ? 1 : 0;
    assign rojo  = ~verde;
endmodule

```

8. Testbench (Bloque de Estímulo)

Es un conjunto de sentencias que genera entradas para probar un diseño digital. También se le conoce como banco de pruebas.

Un módulo especial para probar el diseño. Genera señales de entrada y verifica salidas.

- **No es sintetizable:** Solo sirve para simular.
- **Ejemplo:**

```

module testbench;
    reg  sensor; // Simula el sensor
    wire verde, rojo;

    // Instanciar el semáforo
    semaforo prueba (.sensor(sensor), .verde(verde), .rojo(rojo));

    // Generar estímulos
    initial begin
        sensor = 0; // Sin coches
        #10;        // Esperar 10 unidades de tiempo
        sensor = 1; // Llega un coche
        #20 $finish; // Terminar simulación
    end
endmodule

```

¿Por qué son importantes estos bloques?

Cada bloque cumple un rol específico: los **módulos** organizan el diseño, las **variables** manejan datos, el **flujo de datos** define conexiones, y el **testbench** asegura que todo funcione correctamente. Dominar estos conceptos permite crear circuitos complejos, como procesadores o sistemas de control, de manera estructurada y eficiente.

ESTILOS DE PROGRAMACIÓN EN VERILOG

Verilog permite diseñar circuitos digitales utilizando cuatro niveles de abstracción, cada uno con un enfoque distinto. Estos estilos van desde descripciones generales (como algoritmos) hasta detalles técnicos (como transistores). A continuación, se explican con ejemplos prácticos:

1. Nivel de Comportamiento (Algorítmico)

Describe **qué hace el circuito**, no cómo lo hace. Es el estilo más abstracto y cercano al lenguaje natural.

Características:

- Se usan estructuras como if-else, case y bucles (for, while).
- Ideal para algoritmos complejos o cuando no se conoce la implementación física.
- Difícil de sintetizar automáticamente en hardware.

Ejemplo (Multiplexor 4:1):

```
module mux4a1 (  
    input [3:0] d, // Entradas d0, d1, d2, d3  
    input [1:0] sel, // Selector  
    output reg y // Salida  
);  
    always @(*) begin  
        case(sel)  
            2'b00: y = d[0];  
            2'b01: y = d[1];  
            2'b10: y = d[2];  
            2'b11: y = d[3];  
        endcase  
    end  
endmodule
```



Ilustración 1. Mux 4:1 Nivel de comportamiento

2. Nivel de Flujo de Datos

Describe **cómo se mueven los datos** entre componentes usando operadores lógicos.

Características:

- Usa asignaciones continuas (assign).
- Similar a ecuaciones lógicas.
- Fácil de traducir a hardware.

Ejemplo (Mismo multiplexor 4:1):

```
module mux4a1 (  
    input [3:0] d,  
    input [1:0] sel,  
    output y  
);  
    assign y = (sel == 2'b00) ? d[0] :  
               (sel == 2'b01) ? d[1] :  
               (sel == 2'b10) ? d[2] : d[3];  
endmodule
```

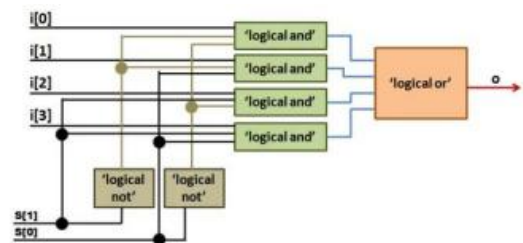


Ilustración 2. Mux 4:1 Nivel de flujo de datos

3. Nivel Estructural (Puertas Lógicas)

Describe el circuito usando **componentes predefinidos** (puertas lógicas) y sus conexiones.

Características:

- Se asemeja a un diagrama esquemático.
- Usa primitivas de Verilog como and, or, not, etc.
- Más cercano al hardware real, pero requiere más líneas de código.

Ejemplo (Mismo multiplexor 4:1):

```
module mux4a1 (  
    input d0, d1, d2, d3,  
    input [1:0] sel,  
    output y  
);  
    wire not_sel0, not_sel1;  
    wire y0, y1, y2, y3;
```

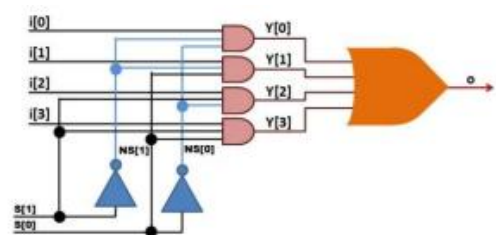


Ilustración 3. Mux 4:1 Nivel de puerta

```

// Invertir señales del selector
not (not_sel0, sel[0]);
not (not_sel1, sel[1]);

// Puertas AND para seleccionar entradas
and (y0, d0, not_sel1, not_sel0);
and (y1, d1, not_sel1, sel[0]);
and (y2, d2, sel[1], not_sel0);
and (y3, d3, sel[1], sel[0]);

// Puerta OR para combinar resultados
or (y, y0, y1, y2, y3);
endmodule

```

4. Nivel de Interruptor (Transistores)

Describe el circuito a nivel de **transistores y conexiones físicas**.

Características:

- Usa primitivas como nmos, pmos, cmos, etc.
- Requiere conocimiento de electrónica.
- Rara vez usado en diseños modernos, excepto en aplicaciones específicas.
- Garantiza una mayor precisión en la simulación de la funcionalidad de diseños complejos.
- Permite un análisis y una verificación exhaustivos.

Ejemplo:

```

module inversor (
    input A,
    output Y
);
    supply1 VDD; // Voltaje positivo
    supply0 GND; // Tierra

    pmos (Y, VDD, A); // Transistor PMOS
    nmos (Y, GND, A); // Transistor NMOS
endmodule

```

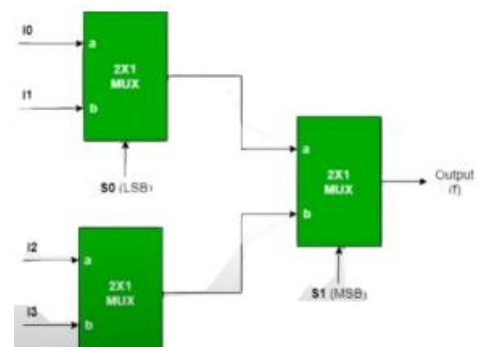


Ilustración 4. Mux 4:1 Nivel de interruptor

5. Modelado de Niveles Mixtos

Verilog permite combinar varios estilos en un mismo diseño. Por ejemplo:

- Un módulo de **comportamiento** puede instanciar otro módulo a nivel **estructural**.
- Un **testbench** (nivel de comportamiento) puede probar un diseño a nivel de **flujo de datos**.

Ejemplo:

```

module sistema (
    input [3:0] datos,
    input [1:0] selector,
    output resultado
);
    // Nivel estructural: Instanciar un multiplexor
    mux4a1 mi_mux (.d(datos), .sel(selector), .y(resultado));
endmodule

```

¿Cuándo usar cada estilo?

Estilo	Ventajas	Casos de Uso
Comportamiento	Rápido para prototipar	Algoritmos complejos, simulaciones
Flujo de Datos	Fácil de sintetizar	Lógica combinacional
Estructural	Precisión en conexiones	Integración de componentes
Interruptor	Control total sobre transistores	Diseño de celdas básicas (ej: SRAM)

Los estilos de programación en Verilog ofrecen flexibilidad para adaptarse a diferentes etapas del diseño. Mientras el **nivel de comportamiento** acelera el desarrollo, el **nivel estructural** garantiza precisión. Elegir el estilo adecuado depende del objetivo: si es simular rápidamente, optimizar recursos o implementar detalles físicos. Combinar estos niveles (modelado mixto) es clave para proyectos complejos y eficientes.

CONCLUSIÓN

Verilog es una herramienta esencial en el diseño de circuitos digitales, ya que permite describir sistemas electrónicos desde conceptos abstractos hasta detalles técnicos específicos. Dominar Verilog y sus estilos de programación no solo facilita la creación de sistemas digitales eficientes, sino que también prepara a los diseñadores para innovar en campos emergentes como la inteligencia artificial, IoT y sistemas embebidos. Esta tarea subraya la importancia de adaptar las herramientas al problema, equilibrando creatividad y rigor técnico.

REFERENCIAS

- edX. (n.d.). **Aprende sobre verilog con cursos online**, edX. Recuperado de <https://www.edx.org/es/aprende/verilog>
- Ruiz, C, V (nov 2012) Introducción a HDL Verilog, Universidad de Sevilla. Recuperado de <https://www.dte.us.es/Members/paulino/Verilog-Intro.pdf>
- pub:vlog [digital Wiki]. (n.d.). **Verilog HDL**, digital Wiki. Recuperado de <http://digital.unex.es/wiki/doku.php?id=pub:vlog>
- **Tutorial Verilog** (n.d.). Recuperado de https://marceluda.github.io/rp_dummy/EEOF2018/Verilog_Tutorial_v1.pdf.
- **Programación en Verilog/Módulos** (n.d.). Wikilibros. Recuperado de https://es.wikibooks.org/wiki/Programaci%C3%B3n_en_Verilog/M%C3%B3dulos
- Admin. (2023, June 2). **Verilog tutorial**. Recuperado de <https://www.chipverify.com/tutorials/verilog>
- **SystemVerilog Data Types**. (n.d.). Recuperado de <https://www.doulos.com/knowhow/systemverilog/systemverilog-tutorials/systemverilog-data-types/>
- Candence (2022, March 17). **Hardware Descripción Idiomas: VHDL vs Verilog, y Sus Usos Funcionales**, Cadence PCB Soluciones. Recuperado de <https://resources.pcb.cadence.com/blog/2020-hardware-description-languages-vhdl-vs-verilog-and-their-functional-uses>